Bài 2.17 – Pipeline & Cross-validation

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *Tìm lỗi trong code*

🎯 Sau bài này bạn sẽ

- Gói toàn bộ quy trình (điền thiếu, chuẩn hóa, one-hot, mô hình) vào **một Pipeline** duy nhất.
- Xử lý cột số và cột chữ khác nhau bằng **ColumnTransformer**.
- Đánh giá mô hình đáng tin cậy bằng **cross-validation**, và đọc được độ dao động giữa các phần.
- Tránh 2 lỗi rò rỉ kinh điển và 1 lỗi chia dữ liệu khiến kết quả sai hoàn toàn.
- Lưu mô hình ra file để dùng lại (chuẩn bị cho Phần C – triển khai).

---

1. Tìm lỗi trong code 🐞

Bạn Tuấn có 100 bệnh nhân, mỗi người đo **10.000 chỉ số gen**. Tuấn chọn 20 gen "liên quan nhất" đến bệnh, rồi kiểm tra bằng cross-validation 5 phần:

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(0)
X_gen = rng.normal(size=(100, 10000))       # 100 người × 10.000 gen: TOÀN SỐ NGẪU NHIÊN
y_benh = rng.integers(0, 2, 100)            # nhãn bệnh: CŨNG NGẪU NHIÊN

chon = SelectKBest(f_classif, k=20).fit(X_gen, y_benh)     # chọn 20 cột liên quan nhất với y
X_20 = chon.transform(X_gen)
print("Độ chính xác:", round(cross_val_score(LogisticRegression(max_iter=1000), X_20, y_benh, cv=5).mean(), 3))
```

Kết quả mẫu:

```
Độ chính xác: 0.87
```

Tuấn hào hứng: "87%! Tìm ra gen gây bệnh rồi!"

Nhưng dữ liệu là **số ngẫu nhiên hoàn toàn**: không gen nào liên quan đến bệnh. Đoán bừa phải chỉ được khoảng 50%. **Lỗi ở đâu?**

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

Bước **chọn gen** đã nhìn thấy **toàn bộ** 100 người, kể cả những người sau đó được dùng làm phần kiểm tra trong cross-validation. Với 10.000 cột ngẫu nhiên, **tình cờ** sẽ có vài cột trùng khớp với y trên đúng 100 người này. Chọn đúng những cột đó rồi kiểm tra trên chính những người đó: tất nhiên trông rất "tốt". Đây là **rò rỉ dữ liệu** (bài 1.11).

Sửa: đưa bước chọn gen **vào trong** pipeline, để nó được làm lại **riêng trên phần train** của mỗi lần chia:

```
from sklearn.pipeline import make_pipeline

dung = make_pipeline(SelectKBest(f_classif, k=20),
LogisticRegression(max_iter=1000))
print("Độ chính xác thật:", round(cross_val_score(dung, X_gen, y_benh,
cv=5).mean(), 3))
```

Kết quả mẫu:

```
Độ chính xác thật: 0.48
```

48%: đúng bằng đoán bừa, đúng với sự thật. Lỗi này **đã thật sự xảy ra** trong nhiều nghiên cứu y sinh học được công bố. Bài này dạy cách để nó **không bao giờ** xảy ra với bạn.

---

2. Pipeline

2.1. Khái niệm

- **Pipeline (Đường ống):** Nối nhiều bước xử lý và mô hình thành **một đối tượng duy nhất**, có `fit` và `predict` như một mô hình bình thường.
- **Ví dụ đời thực:** Dây chuyền rửa xe tự động: xịt nước → bọt xà phòng → chà → xả → sấy. Xe vào đầu này, ra đầu kia. Không ai quên bước nào, không bước nào bị đảo thứ tự.

```
                        Pipeline
   X ──► [ Điền thiếu ] ──► [ Chuẩn hóa ] ──► [ One-hot ] ──► [ Mô hình ] ──► dự
đoán
         (fit trên train)  (fit trên train)  (fit trên train)  (fit trên train)
```

2.2. Pipeline giải quyết 3 vấn đề

Vấn đề nếu làm rời rạc	Pipeline giải quyết
Quên fit scaler/encoder chỉ trên train → rò rỉ	`pipeline.fit(X_train)` tự fit mọi bước chỉ trên train

Vấn đề nếu làm rời rạc	Pipeline giải quyết
Quên áp dụng y hệt các bước cho dữ liệu mới	`pipeline.predict(X_moi)` tự chạy đủ các bước theo đúng thứ tự
Triển khai phải mang theo nhiều đối tượng (scaler, encoder, model)	Lưu **một file** duy nhất

Bạn đã dùng `make_pipeline` từ bài 2.6. Giờ là lúc dùng nó đầy đủ.

---

3. ColumnTransformer: cột số và cột chữ xử lý khác nhau

Dữ liệu tín dụng có 7 cột số và 13 cột chữ. Cột số cần điền thiếu + chuẩn hóa. Cột chữ cần điền thiếu + one-hot.

```
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import roc_auc_score

credit = fetch_openml("credit-g", version=1, as_frame=True).frame
X = credit.drop(columns="class")
y = (credit["class"] == "bad").astype(int)
cot_so = X.select_dtypes("number").columns.tolist()        # tự lấy danh sách cột
số
cot_chu = X.select_dtypes("category").columns.tolist()     # tự lấy danh sách cột
chữ
print(len(cot_so), len(cot_chu), cot_so)

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

tien_xu_ly = ColumnTransformer([
    ("so", Pipeline([("dien", SimpleImputer(strategy="median")),            # điền
thiếu bằng trung vị
                     ("chuan_hoa", StandardScaler())]), cot_so),
    ("chu", Pipeline([("dien", SimpleImputer(strategy="most_frequent")),    # điền
thiếu bằng mode
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
cot_chu),
])
mh = Pipeline([("tien_xu_ly", tien_xu_ly),
               ("mo_hinh", LogisticRegression(max_iter=1000))])
mh.fit(Xtr, ytr)                                                    # MỘT lệnh
fit cho cả dây chuyền
print("AUC test:", round(roc_auc_score(yte, mh.predict_proba(Xte)[:, 1]), 3))
```

(Nhớ `from sklearn.linear_model import LogisticRegression` nếu chạy ô này riêng.)

Kết quả mẫu:

```
7 13 ['duration', 'credit_amount', 'installment_commitment', 'residence_since',
'age', 'existing_credits', 'num_dependents']
AUC test: 0.804
```

Nhìn vào bên trong

```
print(mh.named_steps["tien_xu_ly"].transform(Xtr[:2]).shape)                # sau
tiền xử lý còn bao nhiêu cột
print(mh.named_steps["tien_xu_ly"].get_feature_names_out()[:5].tolist())    # tên
các cột mới
```

Kết quả mẫu:

```
(2, 61)
['so__duration', 'so__credit_amount', 'so__installment_commitment',
'so__residence_since', 'so__age']
```

20 cột gốc → **61 cột** (7 cột số + 54 cột one-hot). Tên cột có tiền tố `so__`, `chu__` theo tên bạn đặt cho từng nhánh.

`handle_unknown="ignore"`: sẵn sàng cho dữ liệu lạ

Ngoài đời, sẽ có ngày xuất hiện một mục đích vay **chưa từng có** trong dữ liệu train:

```
kh_moi = Xte.iloc[[0]].copy()
kh_moi["purpose"] = "crypto"                    # mục đích vay mới, chưa từng thấy
print(mh.predict_proba(kh_moi).round(3))
```

Kết quả mẫu:

```
[[0.758 0.242]]
```

Không lỗi: các cột one-hot của `purpose` đều bằng 0, mô hình vẫn dự đoán dựa trên các thông tin khác. Thiếu `handle_unknown="ignore"`, hệ thống sẽ **sập** khi gặp giá trị lạ.

> 💡 Mọi bước tiền xử lý **có "học" từ dữ liệu** đều phải nằm trong pipeline: điền thiếu (học trung vị), chuẩn hóa (học μ, σ), one-hot (học danh sách nhóm), chọn đặc trưng, PCA, target encoding. Các phép

> biến đổi **không học gì** (ví dụ chia hai cột, lấy log) có thể làm trước, như hàm `them_dac_trung` ở bài 2.16.

---

4. Cross-validation (Kiểm định chéo)

4.1. Vấn đề: một lần chia là "may rủi"

Cùng một mô hình, chỉ đổi cách chia train/test:

```
for seed in [1, 2, 3]:
    Xtr2, Xte2, ytr2, yte2 = train_test_split(X, y, test_size=0.2,
random_state=seed, stratify=y)
    m = Pipeline([("tien_xu_ly", tien_xu_ly), ("mo_hinh",
LogisticRegression(max_iter=1000))]).fit(Xtr2, ytr2)
    print("seed", seed, "→ AUC test:", round(roc_auc_score(yte2,
m.predict_proba(Xte2)[:, 1]), 3))
```

Kết quả mẫu:

```
seed 1 → AUC test: 0.81
seed 2 → AUC test: 0.776
seed 3 → AUC test: 0.789
```

AUC dao động từ 0,776 đến 0,81 **chỉ vì may rủi** của việc chia. Nếu mô hình A đạt 0,79 và mô hình B đạt 0,80 trên một lần chia, bạn **không thể** kết luận B tốt hơn.

4.2. K-Fold Cross-validation

- **Định nghĩa:** Chia dữ liệu train thành **K phần (fold)** bằng nhau. Lần lượt dùng **mỗi phần làm validation một lần**, K−1 phần còn lại để huấn luyện. Được K điểm số → lấy **trung bình** và **độ lệch chuẩn**.
- **Ví dụ đời thực:** Thay vì chấm năng lực học sinh qua **một** đề thi, cho làm **5 đề** khác nhau rồi lấy điểm trung bình.

```
5-Fold Cross-validation:
Lần 1: [VAL ] [train] [train] [train] [train]  → điểm 1
Lần 2: [train] [VAL ] [train] [train] [train]  → điểm 2
Lần 3: [train] [train] [VAL ] [train] [train]  → điểm 3
Lần 4: [train] [train] [train] [VAL ] [train]  → điểm 4
Lần 5: [train] [train] [train] [train] [VAL ]  → điểm 5
                                    Kết quả: trung bình ± độ lệch chuẩn
```

Mỗi mẫu được dùng làm validation **đúng một lần** và làm train K−1 lần. Không lãng phí dữ liệu.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)   # Stratified: giữ tỉ lệ 30% vỡ nợ ở mỗi phần
diem = cross_val_score(mh, Xtr, ytr, cv=cv, scoring="roc_auc")     # CHỈ dùng tập train
print(diem.round(3), "→", round(diem.mean(), 3), "±", round(diem.std(), 3))
```

Kết quả mẫu:

```
[0.808 0.831 0.758 0.676 0.779] → 0.771 ± 0.053
```

Cùng một mô hình, điểm từng phần dao động từ **0,676 đến 0,831**. Một con số duy nhất (như AUC test 0,804 ở mục 3) chỉ là một mẫu trong khoảng dao động đó.

> ⚠️ Cross-validation dùng trên **tập train**. Tập test vẫn là "két sắt" (bài 1.11), chỉ mở một lần cuối cùng.

4.3. Nhiều chỉ số cùng lúc, và so sánh train–validation

```
from sklearn.model_selection import cross_validate

kq = cross_validate(mh, Xtr, ytr, cv=cv, scoring=["roc_auc", "accuracy", "recall"], return_train_score=True)
print({k: round(v.mean(), 3) for k, v in kq.items() if "time" not in k})
```

Kết quả mẫu:

```
{'test_roc_auc': 0.771, 'train_roc_auc': 0.84, 'test_accuracy': 0.738, 'train_accuracy': 0.79, 'test_recall': 0.433, 'train_recall': 0.522}
```

(`test_...` ở đây là điểm trên **phần validation** của mỗi lần chia, không phải tập test thật.) Train AUC 0,84 so với validation 0,771: có khoảng cách, mô hình **hơi overfit** (bài 2.6). Đây là thông tin quý để chọn regularization ở bài 2.18.

4.4. So sánh mô hình đúng cách

```
from sklearn.ensemble import RandomForestClassifier
import lightgbm as lgb

cac_mh = {
    "Logistic": mh,
    "RandomForest": Pipeline([("tien_xu_ly", tien_xu_ly),
                              ("mo_hinh", RandomForestClassifier(n_estimators=300,
```

```
    random_state=42, n_jobs=-1))]),
        "LightGBM": lgb.LGBMClassifier(n_estimators=300, learning_rate=0.03,
    num_leaves=15, subsample=0.8,
                                       subsample_freq=1, colsample_bytree=0.8,
    random_state=42, verbose=-1),  # tự xử lý cột chữ
    }
    for ten, m in cac_mh.items():
        s = cross_val_score(m, Xtr, ytr, cv=cv, scoring="roc_auc")
        print(f"{ten:<13}: AUC CV = {s.mean():.3f} ± {s.std():.3f} | từng phần:
    {s.round(3)}")
```

Kết quả mẫu:

```
Logistic     : AUC CV = 0.771 ± 0.053 | từng phần: [0.808 0.831 0.758 0.676 0.779]
RandomForest : AUC CV = 0.791 ± 0.032 | từng phần: [0.832 0.801 0.775 0.737 0.809]
LightGBM     : AUC CV = 0.783 ± 0.034 | từng phần: [0.823 0.794 0.779 0.722 0.797]
```

Đọc kết quả đúng cách:

- Random Forest có trung bình cao nhất (0,791), nhưng chênh lệch với hai mô hình kia (0,008–0,020) **nhỏ hơn độ dao động** (±0,03–0,05). Không thể khẳng định chắc chắn mô hình nào hơn.
- Để ý **phần thứ 4** luôn thấp nhất ở cả 3 mô hình (0,676; 0,737; 0,722): phần dữ liệu đó "khó" hơn. Vì dùng **cùng** cách chia `cv`, các mô hình được so sánh **công bằng** trên cùng các phần.
- Random Forest vừa có điểm cao nhất vừa **ổn định nhất** (± 0,032) → là ứng viên tốt cho Project 1.

---

5. Lỗi chia dữ liệu: K-Fold không xáo trộn

`cross_val_score(..., cv=5)` (truyền số nguyên) với bài toán **hồi quy** dùng `KFold` **không xáo trộn**: chia theo đúng thứ tự dòng. Dữ liệu nhà California được sắp xếp theo kinh độ (bài 1.11)!

```
from sklearn.model_selection import KFold

df = nap_du_lieu_nha()        # ô chuẩn bị dữ liệu ở bài 2.1
goc = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu", "dan_so",
"so_ho", "thu_nhap"]
lgbm = lgb.LGBMRegressor(n_estimators=500, learning_rate=0.05, random_state=42,
verbose=-1)
for ten, cach_chia in [("cv=5 (không xáo)", 5), ("KFold có xáo", KFold(5,
shuffle=True, random_state=42))]:
    s = cross_val_score(lgbm, df[goc], df["gia_nha"], cv=cach_chia, scoring="r2")
    print(f"{ten:<17}: {s.round(3)} → {s.mean():.3f}")
```

Kết quả mẫu:

```
cv=5 (không xáo) : [-0.074  0.602  0.585  0.574  0.577] → 0.453
KFold có xáo     : [0.816 0.824 0.807 0.827 0.81 ] → 0.817
```

Không xáo trộn: phần đầu tiên toàn là các khu **phía đông** (kinh độ lớn nhất), mô hình phải dự đoán cho vùng nó **chưa từng thấy**, R^2 âm. Cùng một mô hình, cùng dữ liệu, kết luận khác hẳn: 0,453 so với 0,817.

> ⚠️ `cv=5` (số nguyên) dùng `StratifiedKFold` **không xáo trộn** cho phân loại, và `KFold` **không xáo trộn** cho hồi quy. **Luôn truyền một đối tượng có `shuffle=True`**, trừ khi dữ liệu theo thời gian.

Các kiểu chia đặc biệt

Tình huống	Dùng	Lý do
Phân loại	`StratifiedKFold(shuffle=True)`	Giữ tỉ lệ lớp ở mỗi phần
Hồi quy	`KFold(shuffle=True)`	
Dữ liệu theo thời gian (doanh thu, giá cổ phiếu)	`TimeSeriesSplit`	Luôn huấn luyện trên quá khứ, kiểm tra trên tương lai
Nhiều dòng thuộc cùng một đối tượng (nhiều lần khám của cùng bệnh nhân, nhiều đơn của cùng khách)	`GroupKFold`	Không để cùng một người vừa ở train vừa ở validation

6. Lưu và tải mô hình

```python
import joblib

mh.fit(Xtr, ytr)
joblib.dump(mh, "mo_hinh_tin_dung.joblib")          # lưu CẢ pipeline (tiền xử lý + mô hình) vào 1 file
mh_tai_lai = joblib.load("mo_hinh_tin_dung.joblib") # tải lại, ví dụ trong một ứng dụng web
print(mh_tai_lai.predict_proba(Xte[:2]).round(3))   # dùng ngay với dữ liệu thô
```

Kết quả mẫu:

```
[[0.773 0.227]
 [0.899 0.101]]
```

File chỉ khoảng 8 KB. Ứng dụng web chỉ cần tải file này và gọi `predict_proba` với dữ liệu thô của khách: mọi bước điền thiếu, chuẩn hóa, one-hot tự chạy bên trong. Bạn sẽ dùng chính cách này để dựng API ở bài C.2.

> ⚠️ Chỉ tải file `.joblib` từ nguồn **bạn tin tưởng**: file này có thể chứa mã độc chạy ngay khi tải. Và dùng **cùng phiên bản** Scikit-learn khi lưu và khi tải.

7. Quy trình chuẩn từ nay về sau

```
1. Chia train / test (test cất vào "két sắt")
2. Viết hàm tạo đặc trưng không-học (tỉ lệ, log...) → áp dụng cho cả hai
3. Dựng Pipeline: ColumnTransformer (tiền xử lý) + mô hình
4. So sánh các mô hình bằng cross-validation TRÊN TRAIN (có shuffle!)
5. Tinh chỉnh tham số bằng cross-validation (bài 2.18)
6. Fit mô hình cuối cùng trên TOÀN BỘ train
7. Đánh giá trên test MỘT LẦN → báo cáo
8. Lưu pipeline bằng joblib
```

8. Bài tập

1. **Tìm lỗi:** đoạn code sau có rò rỉ dữ liệu không? Sửa thế nào?

```
X_s = StandardScaler().fit_transform(X_so)
print(cross_val_score(LogisticRegression(), X_s, y, cv=5).mean())
```

2. Thêm bước `PCA(n_components=0.95)` vào nhánh cột số của `tien_xu_ly`. AUC CV thay đổi thế nào?
3. Dùng `cross_validate` cho Random Forest ở mục 4.4 với `return_train_score=True`. Khoảng cách train–validation lớn hay nhỏ? So với Logistic Regression?
4. Bạn có dữ liệu 5.000 lần khám của 800 bệnh nhân (mỗi người khám nhiều lần). Nên dùng kiểu chia nào? Chuyện gì xảy ra nếu dùng `KFold` thường?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. **Có**, rò rỉ nhẹ: scaler học μ, σ từ toàn bộ dữ liệu, kể cả phần sẽ làm validation trong mỗi lần chia. Ngoài ra `cv=5` không xáo trộn. Sửa:

```
mh_dung = make_pipeline(StandardScaler(), LogisticRegression())
print(cross_val_score(mh_dung, X_so, y, cv=StratifiedKFold(5, shuffle=True,
random_state=42)).mean())
```

2. Code:

```
from sklearn.decomposition import PCA
tien_xu_ly_pca = ColumnTransformer([
    ("so", Pipeline([("dien", SimpleImputer(strategy="median")), ("chuan_hoa",
StandardScaler()),
                     ("pca", PCA(n_components=0.95))]), cot_so),
    ("chu", Pipeline([("dien", SimpleImputer(strategy="most_frequent")),
```

```
                        ("onehot", OneHotEncoder(handle_unknown="ignore"))]),
cot_chu),
])
m = Pipeline([("tien_xu_ly", tien_xu_ly_pca), ("mo_hinh",
LogisticRegression(max_iter=1000))])
print(cross_val_score(m, Xtr, ytr, cv=cv, scoring="roc_auc").mean().round(3))
```

Với chỉ 7 cột số ít tương quan với nhau, PCA thường không giúp, có thể giảm nhẹ. Ghi lại kết quả thật. 3. Random Forest có train AUC **gần 1,0** (các cây mọc tối đa học thuộc train), khoảng cách train–validation **lớn hơn nhiều** so với Logistic (0,84 / 0,771). Nhưng như bài 2.10, với Random Forest điều đó không có nghĩa là tệ: điểm **validation** mới là thứ quan trọng. 4. Dùng `GroupKFold` với nhóm là mã bệnh nhân. Nếu dùng `KFold` thường, các lần khám của **cùng một người** sẽ nằm cả ở train lẫn validation; mô hình "nhận ra người quen" thay vì học quy luật bệnh, điểm validation **cao giả tạo**, và mô hình sẽ thất bại với bệnh nhân mới.

---

9. Flashcard

Mặt trước	Mặt sau
Pipeline giải quyết gì?	Tránh rò rỉ, không quên bước, triển khai bằng một đối tượng.
ColumnTransformer dùng để làm gì?	Áp dụng xử lý khác nhau cho các nhóm cột khác nhau (số / chữ).
`handle_unknown="ignore"`?	One-hot không báo lỗi khi gặp nhóm chưa từng thấy.
Bước nào phải nằm trong pipeline?	Mọi bước "học" từ dữ liệu: điền thiếu, chuẩn hóa, encoding, chọn đặc trưng, PCA.
K-Fold CV làm gì?	Chia train thành K phần, mỗi phần làm validation một lần, lấy trung bình ± độ lệch chuẩn.
Vì sao cần CV thay vì một lần chia?	Một lần chia có nhiều may rủi; CV cho ước lượng ổn định hơn và cho biết độ dao động.
`cv=5` (số nguyên) có xáo trộn không?	Không! Luôn dùng `KFold`/`StratifiedKFold` với `shuffle=True`.
Dữ liệu thời gian dùng kiểu chia nào?	`TimeSeriesSplit`.
Nhiều dòng cùng một người dùng kiểu chia nào?	`GroupKFold`.
Lưu mô hình thế nào?	`joblib.dump(pipeline, "file.joblib")`; tải bằng `joblib.load`.

Bài tiếp theo: 2.18 – Tinh chỉnh tham số: Grid Search, Random Search.